

Talent-X

Document 4 — Déploiement, exploitation, tests et scalabilité

Cible d'hébergement OVHcloud, chaîne de livraison et stratégie de montée en charge — révision intégrant l'audit de conception.

Référence	TX-OPS-004
Version	2.0
Date	4 juin 2026
Statut	Document technique détaillé — révision 2.0 (corrections d'audit + améliorations)

Ce document fait partie du lot documentaire Talent-X. Il décrit le déploiement, l'exploitation, les tests et la scalabilité. Il s'appuie sur l'architecture (TX-ARCH-001, dont la vue C4 et les exigences non fonctionnelles §10), la sécurité (TX-SEC-003) et le modèle de données (TX-DATA-006).

Historique des révisions

Version	Date	Nature des modifications
1.0	8 avril 2026	Version initiale.
2.0	4 juin 2026	Ajout du service worker (BullMQ) aux environnements, à l'architecture et aux paliers ; étape de migration gâtée (expand-contract, Prisma/TypeORM) et tests de contrat (Pact) dans le pipeline ; tests de charge (k6/Gatling/Locust) ; observabilité managée au MVP ; supervision de la file de jobs ; SLO et budget d'erreur ; garde des clés de sauvegarde ; modèle de coûts indicatif ; feature flags et politique OTA ; réconciliation de la charge en utilisateurs actifs et de la tension disponibilité / mononœud avec TX-ARCH-001 ; nouveaux risques et décisions à valider.

Sommaire

Historique des révisions	2
1. Objectifs d'exploitation	3
2. Environnements recommandés	3
3. Architecture cible OVHcloud	3
4. Organisation du déploiement	4
4.1 MVP opérationnel	4
4.2 Montée en charge intermédiaire	4
4.3 Cible avancée	4
5. Pipeline CI/CD recommandé	4
6. Stratégie de test	5
7. Monitoring et observabilité	5
7.1 Niveaux d'alerte indicatifs	6
7.2 SLO et budget d'erreur	6
8. Sauvegarde, restauration et continuité	6
9. Scalabilité par phases	6
10. Modèle de coûts indicatif	7
11. Feature flags et gestion des releases	7
12. Runbook minimal d'exploitation	8
13. Feuille de route opérationnelle	8
14. Risques opérationnels et mitigations	8
15. Décisions à valider	8
16. Conclusion	9

1. Objectifs d'exploitation

Le socle d'exploitation de Talent-X doit rester simple au démarrage, tout en permettant une progression sans rupture vers une architecture plus distribuée. Le bon niveau de sophistication n'est pas celui d'un grand groupe dès le premier jour, mais celui d'une startup sérieuse : reproductible, observable, sécurisée et extensible.

- Automatiser le maximum de tâches répétitives : build, tests, migrations, déploiement, vérifications post-déploiement.
- Rendre l'état du système visible : logs, métriques, alertes, health checks.
- Disposer d'un support d'exécution fiable pour les traitements asynchrones (exports RGPD, purges planifiées, notifications).
- Relier les alertes à des objectifs de niveau de service (SLO) plutôt qu'à des seuils isolés.
- Garantir une restauration réaliste grâce à des sauvegardes testées.
- Limiter les points uniques de défaillance dès que la traction produit le justifie.

2. Environnements recommandés

Environnement	Rôle	Composition minimale	Usage
Développement	Itération locale	App mobile + API + worker + PostgreSQL + Redis en Docker Compose	Développement quotidien.
Staging	Pré-production	Réplique simplifiée de la prod (API, worker, base, cache, observabilité)	Validation fonctionnelle, migrations et E2E.
Production	Service live	Reverse proxy, API, worker(s), base, cache, stockage objet, observabilité	Exploitation utilisateurs réels.

Le **worker** est un process séparé de l'API qui consomme la file de jobs Redis (BullMQ), conformément à TX-ARCH-001 §4.5. Le staging ne doit pas être négligé : il valide la chaîne de livraison, les migrations, les tests bout en bout et le comportement de l'application mobile contre une infrastructure réaliste.

3. Architecture cible OVHcloud

La production peut démarrer sur une infrastructure modeste, mais le découpage logique reste identique afin de faciliter l'évolution. Le schéma de conteneurs de référence figure dans TX-ARCH-001 (vue C4).

- Un reverse proxy Nginx expose l'API et applique les premières mesures de sécurité réseau.
- Les conteneurs applicatifs NestJS sont déployés à partir d'images versionnées.
- Un ou plusieurs workers consomment la file de jobs pour les traitements asynchrones (exports, purges, notifications).
- PostgreSQL reste la source de vérité métier ; Redis joue un double rôle : cache technique et file de jobs.
- Le stockage objet héberge les fichiers temporaires, avatars et exports selon la politique retenue.

- Les notifications sortent vers APNs et FCM, hors UE : ce transfert est encadré et minimisé (cf. TX-SEC-003 §10).

4. Organisation du déploiement

4.1 MVP opérationnel

- 1 VPS ou instance dédiée suffisamment dimensionnée.
- Docker Compose pour le reverse proxy, l'API, le worker, PostgreSQL et Redis.
- Sauvegardes externes chiffrées et supervision de base obligatoires.
- Convient au lancement initial et à la validation marché si le trafic reste modéré.

Disponibilité au MVP. Un nœud unique est un point de défaillance unique. Viser une disponibilité best-effort de l'ordre de 99 % est réaliste à ce stade ; l'objectif de 99,5 % et les déploiements sans interruption supposent au moins deux instances d'API derrière Nginx et des migrations rétrocompatibles (cf. §4.2 et TX-ARCH-001 §10). Cet écart doit être assumé explicitement, pas masqué.

4.2 Montée en charge intermédiaire

- Séparer la base de données sur une instance dédiée ou managée.
- Passer à au moins deux réplicas d'API derrière Nginx ou un load balancer ; répliquer aussi le worker selon la charge des jobs.
- Adopter des migrations expand-contract (rétrocompatibles) pour permettre le déploiement sans interruption.
- À partir de deux instances d'API, viser 99,5 % de disponibilité et le zéro-downtime.
- Renforcer la surveillance des performances, du pool de connexions et de la file de jobs.

4.3 Cible avancée

- Kubernetes managé ou orchestrateur équivalent si la complexité d'exploitation est justifiée par la charge.
- Horizontal Pod Autoscaler côté API ; workers mis à l'échelle indépendamment selon la profondeur de file.
- PgBouncer pour le pooling ; réplication PostgreSQL pour soulager les lectures.
- Séparation plus nette entre workloads synchrones, tâches planifiées et traitements batch.

5. Pipeline CI/CD recommandé

Le pipeline ne sert pas seulement à déployer vite : il empêche qu'une régression de sécurité, de contrat d'API ou de migration de schéma atteigne la production sans filet.

- Étape 1 — Contrôles statiques : lint, formatage, typage.
- Étape 2 — Tests unitaires backend et frontend avec seuils de couverture.
- Étape 3 — Tests d'intégration sur services conteneurisés.
- Étape 4 — Tests de contrat mobile↔API (Pact) : vérifier la compatibilité du contrat.
- Étape 5 — Scans de dépendances et de sécurité.
- Étape 6 — Build Docker multi-stage et publication dans le registry.
- Étape 7 — Déploiement automatique vers staging.

- Étape 8 — Migration de schéma gâtée (Prisma ou TypeORM), en mode expand-contract, vérifiée sur staging.
- Étape 9 — Tests E2E et validations manuelles ciblées.
- Étape 10 — Promotion contrôlée vers production, migration appliquée selon la même discipline.

Migration gâtée. La migration de schéma est une étape explicite et bloquante — c'est un vecteur d'incident classique en production. Les migrations sont rétrocompatibles (expand-contract) afin de ne pas casser les instances encore en version précédente pendant un déploiement progressif. L'outil (Prisma ou TypeORM) reste à arrêter (section 15).

6. Stratégie de test

Niveau	Outils possibles	Cible	Exemples
Unitaires	Jest	Services, utilitaires, stores	Règles métier, calculs, guards.
Intégration	Jest + Supertest	Contrôleurs, DB, Redis	Login, CRUD séance, gestion d'accès.
Contrat	Pact	Interface mobile↔API	Protéger la compatibilité ascendante du contrat.
E2E mobile	Detox ou Maestro	Parcours critiques	Inscription, connexion, création de séance, saisie de performance.
Charge / performance	k6, Gatling ou Locust	Parcours critiques sous charge	Avant chaque palier : valider latence p95 et capacité.
Sécurité	OWASP ZAP, scans de dépendances	API et supply chain	Tests mensuels ou à chaque release majeure.
Restauration	Scripts de backup/restore	Opérations	Test périodique de restauration PostgreSQL.

Les tests liés au RGPD et à la sécurité ne doivent pas être oubliés : export complet, suppression effective, rotation et détection de réutilisation des tokens, contrôle des droits croisés (matrice d'autorisation), et vérification des journaux.

7. Monitoring et observabilité

- Logs structurés JSON avec corrélation ID par requête.
- Métriques infrastructure : CPU, mémoire, disque, réseau, saturation des conteneurs.
- Métriques applicatives : taux d'erreur, latence p95, volume d'appels, connexions actives, échecs d'authentification.
- Métriques de la file de jobs : profondeur de la file, jobs en échec, latence de traitement, jobs rejoints.
- Health checks dédiés pour l'API, la base, Redis et les dépendances critiques.
- Tableaux de bord séparés pour exploitation, sécurité et performance produit.

Observabilité au MVP. Une observabilité managée (logs et métriques hébergés) réduit la charge d'exploitation d'un solo. L'internalisation (Prometheus + Grafana + Loki) peut venir plus tard, à charge constante — cohérent avec le principe « proportionné » du document et

avec TX-ARCH-001 (ADR-11). L'essentiel est moins le nom des outils que la discipline : métriques utiles, seuils documentés, alertes actionnables.

7.1 Niveaux d'alerte indicatifs

Niveau	Déclencheurs
Critique	Health check KO sur plusieurs vérifications successives, indisponibilité API, erreur de migration, file de jobs bloquée ou jobs en échec en série, saturation disque sévère.
Haute	Taux d'erreur > 5 %, latence p95 > 1 s, profondeur de file anormale, pic d'échecs de connexion, hausse brutale des 429 ou 401.
Moyenne	Hausse CPU soutenue, croissance anormale du volume de logs, allongement des temps de requête SQL.

7.2 SLO et budget d'erreur

- Définir des SLO simples reliés aux exigences non fonctionnelles (TX-ARCH-001 §10) : disponibilité et latence p95.
- Suivre un budget d'erreur : lorsqu'il est consommé, prioriser la fiabilité sur les nouvelles fonctionnalités.
- Les alertes du §7.1 découlent du taux de consommation du budget (burn rate), pas de seuils arbitraires.

Indicateur	Objectif (à confirmer)	Source
Disponibilité API (mensuelle)	~99 % au MVP ; 99,5 % à partir de la Phase 2	Health checks
Latence p95 (lecture)	< 300 ms	Métriques applicatives
Taux d'erreurs 5xx	< 1 %	Métriques applicatives

8. Sauvegarde, restauration et continuité

- Sauvegardes PostgreSQL automatisées, chiffrées et stockées hors du nœud principal.
- Clés de chiffrement des sauvegardes conservées hors du nœud et hors du même stockage que les backups ; rotation organisée. Une compromission du nœud ne doit pas compromettre les sauvegardes (cf. TX-SEC-003 §12).
- Politique de rétention roulante alignée avec les contraintes RGPD et les besoins métier.
- Vérification régulière de la capacité de restauration, pas seulement de la capacité de sauvegarde.
- Journal d'exécution des backups et alertes en cas d'échec.
- Document court de PRA/PCA décrivant les étapes de reprise minimales.

Le scénario le plus dangereux n'est pas l'absence totale de sauvegarde, mais la croyance erronée que l'on sait restaurer alors qu'aucun exercice n'a été réalisé.

9. Scalabilité par phases

Les volumes sont exprimés en utilisateurs actifs (cf. TX-ARCH-001 §10, qui distingue utilisateurs actifs mensuels, utilisateurs concurrents et RPS cible). Avant chaque passage de palier, valider la capacité par un test de charge (§6) plutôt que de se fier aux seules hypothèses.

Phase	Volume visé	Architecture	Déclencheur de passage
Phase 1	< 500 actifs	Mononœud robuste (API + worker + base + cache)	Lancement MVP et validation marché.
Phase 2	500 à 5 000	API et worker répliqués + base séparée + meilleure supervision	Croissance du trafic et besoins de disponibilité.
Phase 3	5 000 à 50 000	Orchestration, réplicas DB, cache affiné, pooling	Charge régulière et contraintes d'exploitation.
Phase 4	> 50 000	Services segmentés, analytics dédiés, offline avancé	Complexité produit et usage massif.

Le critère de passage s'appuie sur les métriques : saturation CPU, contention base, taux d'erreur, temps de réponse, profondeur de file, coût d'exploitation et fréquence des incidents.

10. Modèle de coûts indicatif

Avertissement. Ordres de grandeur destinés à l'arbitrage, hors taxes et hors coûts annexes (domaine, email transactionnel, stockage objet, gros volumes de sauvegarde, observabilité managée). Les montants évoluent : à confirmer sur la grille tarifaire OVHcloud à jour. ⚠

Phase	Architecture	Coût mensuel (ordre de grandeur)	Facteurs
Phase 1	1 VPS + sauvegardes + observabilité managée	Quelques dizaines d'euros	Dimensionnement du VPS.
Phase 2	≥ 2 instances API + base séparée/managée + cache	De l'ordre de la centaine à quelques centaines d'euros	Nombre de réplicas, type de base.
Phase 3	Orchestration + réplicas DB + pooling	Plusieurs centaines d'euros et plus	Charge, redondance, réplication.
Phase 4	Services segmentés, analytics dédiés	Variable / sur devis	Architecture spécifique.

11. Feature flags et gestion des releases

- Découpler le déploiement (livrer le code) de l'activation (exposer la fonctionnalité) au moyen de feature flags.
- Permettre des déploiements progressifs et des bascules rapides en cas de problème, sans nouveau déploiement.
- Politique OTA (Expo) : distinguer ce qui passe en OTA (bundle JS) de ce qui exige une revue store (changements natifs), avec une stratégie de rollback (cf. TX-ARCH-001, partie mobile).
- Au MVP, un mécanisme simple (table de flags ou service léger) suffit ; pas de plateforme dédiée tant que le besoin ne le justifie pas.

12. Runbook minimal d'exploitation

- Quotidien : vérifier l'état de santé global, les erreurs critiques, l'espace disque et l'état de la file de jobs (profondeur, échecs).
- Hebdomadaire : suivre les latences API, les slow queries et le volume des logs.
- Mensuel : contrôler la restauration d'une sauvegarde sur un environnement isolé.
- Avant chaque release : vérifier migrations expand-contract validées sur staging, variables d'environnement, certificats, jobs planifiés, état des feature flags et rollback.
- Après chaque release : contrôler les health checks, la latence, les erreurs 5xx, la file de jobs et les parcours clés.

13. Feuille de route opérationnelle

- MVP : hébergement simple, worker, supervision de base, pipeline CI/CD avec migration gâtée, sauvegardes, export et effacement opérationnels.
- V2 : dashboards métier plus riches, notifications, duplication de séances, tests de charge, renforcement de la disponibilité (≥ 2 instances).
- V3 : analytics avancés, offline plus poussé, modèle de monétisation, évolution d'architecture sur les requêtes complexes.

14. Risques opérationnels et mitigations

Risque	Probabilité	Impact	Réponse
Bottleneck développeur unique	Élevée	Élevé	Automatisation, documentation, modularité, priorisation stricte.
Dette d'exploitation	Moyenne	Élevé	Supervision et sauvegardes dès le début.
Scope creep	Élevée	Élevé	Protéger le MVP par des frontières fonctionnelles claires.
File de jobs engorgée / worker en panne	Moyenne	Élevé	Supervision de la file, alertes, dimensionnement, reprise des jobs.
Migration non rétrocompatible	Moyenne	Critique	Discipline expand-contract, étape gâtée, test sur staging, rollback.
Incident OVH / infra	Faible à moyenne	Élevé	Backups, scripts d'infra, plan de reprise documenté.
Régression de sécurité	Moyenne	Critique	CI, scans, revue de code, contrôles automatiques.
Surcharge base de données	Moyenne	Élevé	Indexation, requêtes maîtrisées, lecture répliquée si nécessaire.

15. Décisions à valider

Sujet	Recommandation par défaut	Réf.
Outil de migration	Prisma ou TypeORM ; à choisir	5
Observabilité au MVP	Managée, puis internalisée à charge constante	7
Outil de tests de charge	k6 (ou Gatling / Locust)	6

Sujet	Recommandation par défaut	Réf.
Mécanisme de feature flags	Simple au MVP (table ou service léger)	11
Valeurs de SLO	À confirmer avec TX-ARCH-001 §10	7.2
Hébergement base en Phase 2	Instance dédiée ou base managée OVH	4.2
Budget cible par phase	À confirmer sur la grille OVH à jour	10

16. Conclusion

Le meilleur dispositif d'exploitation pour Talent-X reste un dispositif proportionné : assez simple pour être tenu correctement par une personne, assez outillé pour inspirer confiance, et suffisamment modulaire pour supporter la croissance. Cette révision comble les angles morts qui auraient coûté cher en production : support d'exécution des traitements asynchrones, migrations gâtées et rétrocompatibles, tests de charge avant chaque palier, SLO reliant les NFR aux alertes, et garde des clés de sauvegarde.

L'ambition startup-grade ne conduit pas à sur-ingénierer dès le premier jour, mais à préparer proprement les points d'extension qui compteront réellement — et à assumer explicitement la tension entre un MVP mononœud et les objectifs de disponibilité, plutôt que de la laisser implicite.